

STRATEGY DESIGN PATTERN

- ① Definition
- ② Where to use Strategy Design Pattern
- ③ Example of Strategy Design Pattern
- ④ How to use Strategy Design Pattern
- ⑤ Difference between Adapter and Strategy design pattern.
- ⑥ Practical of Strategy Design Pattern

Defⁿ

- ① Behavioral Design Pattern
- ② The strategy design pattern is to define a set of algorithms, encapsulate them in their classes, and make them interchangeable within context objects.

Where to use?

→ If we have functionality which can be implemented in different ways, then we can use strategy design pattern.

Example

```
class GoogleMapClient {  
    Run | Debug  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        String mode = sc.next();  
  
        if(mode.equals(anObject: "car")) {  
            // Logic to find Car path  
            // 100 lines of code  
        } else if(mode.equals(anObject: "walk")) {  
            // Logic to find Car path  
            // 100 lines of code  
        } else {  
            throw new IllegalArgumentException(s: "Invalid Input");  
        }  
    }  
}
```

Problems

- Too many if-else
- Hard to extend
- Violates
 - SRP
 - OCP

How to use strategy design pattern

Client

Google Map Path Strategy

→ findPath()

Context

Walk Mode

→ findPath()

Car Mode

→ findPath()

Strategy

VS

Adapter

① Behavioral

②

If we have functionality which can be implemented in different ways, then we can use strategy design pattern.

① Structural

②

If there is a requirement where you need to integrate 3rd party SDK or library then we can use adapter design pattern.